HOOX TRADING PLATFORM

DEVOPS INFRASTRUCTURE, BINDINGS & SYSTEM RUNBOOKS

Generated: 2026-05-19

Classification: Technical Documentation Spec

Design Style: Monospace Terminal Console layout

TABLE OF CONTENTS

- DEVOPS MANUAL
- OPERATIONS & RUNBOOK
- CLOUDFLARE WORKERS SETUP FLOW
- TUI CODE ARCHITECTURE
- SYSTEM TOPOLOGY & OVERVIEW
- ISOLATE COMMUNICATION SPEC
- SYSTEM DATA ROUTING SPEC
- INFRASTRUCTURE BINDINGS INDEX
- STORAGE & DATA ENGINEERING SPEC
- INTERNAL ENDPOINTS MAP
- VISUAL TOKENS & DESIGN SYSTEM
- HOOX GATEWAY ISOLATE PROFILE
- TRADE-WORKER ISOLATE PROFILE
- AGENT-WORKER ISOLATE PROFILE
- TELEGRAM-WORKER ISOLATE PROFILE
- D-WORKER ISOLATE PROFILE
- WEB-WALLET-WORKER ISOLATE PROFILE
- EMAIL-WORKER ISOLATE PROFILE
- ANALYTICS-WORKER ISOLATE PROFILE
- REPORT-WORKER ISOLATE PROFILE
- NEXT.JS DASHBOARD & OPENNEXT ISOLATE
- PRODUCTION DEPLOYMENT RUNBOOK
- CI/CD PIPELINE AUTOMATION
- OBSERVABILITY & TELEMETRY
- ZERO TRUST & SECURITY HARDENING
- LOCAL DEVELOPMENT SETUP
- TESTING FRAMEWORK & QA STANDARDS
- EDGE DEBUGGING & TELEMETRY
- API ENDPOINT DIRECTORY
- REQUEST PAYLOAD SCHEMAS
- STANDARD RESPONSE SCHEMAS
- CLI ARCHITECTURE & FEATURES

. SHARED `ERRORS` FACTORY MIDDLEWARE

To enforce this response contract without writing redundant JSON wrappers in every worker, we use the standardized `Errors` factory from the shared monorepo package `@jango-blockchained/hoox-shared/errors`:

```
import { Errors } from "@jango-blockchained/hoox-shared/errors";

// . Validation Failures ( Bad Request)
if (!payload.symbol) {
  return Errors.badRequest("Symbol is a required parameter.");
}

// . Secret Key Mismatch ( Unauthorized)
if (requestHeaderKey !== env.INTERNAL_KEY_BINDING) {
  return Errors.unauthorized("Access Denied: Invalid X-Internal-Auth-Key.");
}

// . System Exceptions ( Internal Server Error)
try {
  await database.write(data);
} catch (err: any) {
  return Errors.internal(`Database write exception: ${err.message}`);
}
```

These factory methods automatically serialize the exception, attach the active `requestId` from the context, set the correct HTTP status code, and return a compliant `Response` object.

NEXT STEPS

- [API Endpoint Directory](endpoints.md) - Analyze REST routes, headers, and endpoints mappings.
- [Request Payload Schemas](payloads.md) - Check JSON request payload specifications and typescript interfaces.